JAVA PROGRAMMING

Chapter 1: Write Your First Java Program

By Akanshu Jamwal

Table of Contents

1. Your First Java Program

Your First Complete Program:

```
public class Main {

    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Output: Hello, World!

2. Why 'Hello, World!'?

The Hello, World! program is traditionally used to introduce beginners to a new programming language. It helps you understand the basic structure of a program, how to write and run code, and how output works.

This program helps you understand:

• Basic structure of a program

• How to write and run code

• How output works

Don't worry if you don't understand every line yet. We will break it down step by step. For now, type the program exactly as shown and run it.

3. Understanding the Program

public class Main {

• Every Java program must be inside a class
• Here, Main is the name of the class

public static void main(String[] args)

• This is the entry point of the program
• The JVM starts execution from the main method

System.out.println("Hello, World!");

- System.out.println() prints output to the console
- The text to print must be inside double quotes
- Every statement ends with a semicolon (;)

4. Important Rules to Remember

When using System.out.println():

✓ Everything to be printed goes inside parentheses ( )
✓ The text must be inside double quotes " "
✓ Each statement must end with a semicolon ;

If you violate these rules, your program will produce errors.

5. Basic Structure of a Java Program

```
class Main {
    public static void main(String[] args) {

        // Your code goes here

    }
}
```

Note: You will write your logic inside the main method.

6. Java Comments

As programs grow larger, it becomes important to explain parts of your code. Comments help you document your code effectively.

Key Facts about Comments:

• Comments are notes written inside the code to make it easier to understand

• Comments are ignored by the Java compiler

• Comments are useful for documentation and debugging

Example with Comments:

```
class HelloWorld {
    public static void main(String[] args) {
        // print Hello World to the screen
        System.out.println("Hello World");
    }
}
```

Here, "// print Hello World to the screen" is a comment. The compiler ignores everything after // on that line.

7. Single-Line Comments

A single-line comment starts with **//** and everything after it on that line is ignored by the compiler.

```
// declare and initialize variables
int a = 1;
int b = 3;

// print the output
System.out.println("This is output");
```

8. Multi-Line Comments

If you want to write comments across multiple lines, use **/* */** syntax. Everything between /* and */ is ignored by the compiler.

```
/*
   This is a multi-line comment.
   It can span multiple lines.
*/

class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

9. Using Comments for Debugging

Sometimes, while testing your program, you may want to temporarily disable certain lines of code. Instead of deleting problematic lines, you can comment them out.

Original Code:

```
public class Main {
    public static void main(String[] args) {
        System.out.println("codehelp one");
        System.out.println("codehelp two");
        System.out.println("codehelp three");
    }
}
```

After Commenting Out (Debugging):

```
public class Main {
    public static void main(String[] args) {
        System.out.println("codehelp one");
        // System.out.println("codehelp two");
        System.out.println("codehelp three");
    }
}
```

This is a common debugging technique. It allows you to quickly test which line might be causing issues.

10. Why Use Comments?

Comments help in:

• Improving code readability

• Making collaboration easier with other developers

• Debugging programs effectively

• Explaining complex logic

■ **Best Practices to Avoid:**

• Comments should not replace poorly written code

• Always aim to write clean, understandable code first

✓ **Good Practice:**

• Use comments to explain **WHY** something is done — not HOW it is done

• Write self-explanatory code with meaningful variable names

11. Key Takeaways & Summary

Concept	Description
Class	Every Java program must be inside a class
main() method	Entry point of the program; JVM starts execution here
System.out.println()	Prints output to the console
Single-line comment	Starts with //; rest of line is ignored
Multi-line comment	Between /* and */; all content ignored

Quick Checklist:

✓ Every Java program must have a class and a main method

✓ Use System.out.println() to print output

✓ All statements must end with a semicolon (;)

✓ Use // for single-line comments

✓ Use /* */ for multi-line comments

✓ Comments help explain your code but shouldn't replace clean code

✓ Practice by writing and running simple programs

■ What's Next?

Now that you've written your first Java program and learned about comments, the next step is to understand:

- Variables and Data Types
- Operators
- Control Statements (if-else, loops)
- Functions and Methods

Remember: Programming becomes powerful when theory meets execution. Keep practicing consistently!